

TrailSense Multilateration Library: Using the Output

July 12, 2026

CONTENTS

TrailSense Multilateration Library: Using the Output

Using the Output

1. Where the boundary is

2. The output contract (briefly)

The three things consumers most often get wrong

3. The library handles / You are responsible for

4. The consumer responsibilities checklist

5. “No position” and “low confidence”: detect and do

6. Before you go live: integration checklist

TrailSense Multilateration Library: Using the Output

What you build in your own system to consume the positions the library produces. Part of the TrailSense Multilateration Library documentation set.

2. The output contract (briefly)

Per position, exactly these fields (from `ts_position_t` in `include/trailsense_triangulate.h`):

- `mac_hash` (4 bytes) and `band` (0 wifi, 1 ble, 2 cell): which device and radio.
- `lat`, `lon` in degrees. Order: **lat first, then lon**.
- `accuracy_m`: weighted CEP in meters. For a 2-node fix this equals `error_semi_major_m`.
- `confidence`: one of exactly `40` (2 anchors, approximate bilateration), `70` (exactly 3 anchors), `85` (4 or more anchors).
- `measurement_count`: number of participating anchors, NOT the post-outlier survivor count.
- `peak_rssi`: strongest participant RSSI in dBm.
- Error ellipse: `error_semi_major_m`, `error_semi_minor_m`, `error_major_axis_deg` (compass bearing of the semi-major axis, 0 = North, clockwise, folded into `[0,180)`).

The CLI adds two derived fields not in the C struct: `signal_type` (`wifi` / `bluetooth` / `cellular`) and `fingerprint_hash` (band prefix plus hex `mac_hash`, e.g. `w_deadbeef`). Full field-by-field contract: the Output Reference.

THE THREE THINGS CONSUMERS MOST OFTEN GET WRONG

1. **No timestamp in the output.** The library has no clock and never stamps a time. To know when a fix happened, YOU attach your own receive/observation time on the way out (see section 4, “Stamp time”).
2. **The 2-node ellipse cross axis is a mirror-ambiguity bound, not a confidence radius.** For a confidence-40 fix the along-baseline axis is a genuine roughly 1-sigma range uncertainty, but the across-baseline axis is a deterministic hard bound on how far the true position could be mirrored across the anchor baseline. Do not draw it as a 1-sigma circle or feed it to a Gaussian filter as one (see section 4, “Render on a map”).
3. **The library emits lat then lon.** Many mapping/GeoJSON stacks expect `[lon, lat]`; if yours does, swap on handoff. Getting this wrong silently plots every fix in the wrong hemisphere.

3. The library handles / You are responsible for

The library handles	You are responsible for
Correlating observations by <code>(mac_hash, band)</code>	Assembling the input batch and owning the anchor table
Keeping the latest sample per node within a group	Deciding which observations form one batch (windowing)
RSSI to distance via the path-loss model	Persisting the positions that come out
Selecting method by distinct anchor count (2 vs 3+)	Stamping each result with your own time
The solve (bilateration or robust IRLS)	Aging out stale fixes before acting
The position: <code>lat</code> , <code>lon</code>	Correlating fixes over time into tracks
<code>accuracy_m</code> , <code>confidence</code> , <code>measurement_count</code> , <code>peak_rssi</code>	Thresholding on confidence and accuracy for your use case
The error ellipse (with the mixed 2-node semantics)	Rendering on a map, honestly, including the ellipse
A pure, reentrant, allocation-free call	Alerting, acting, deduping, monitoring for no-output

4. The consumer responsibilities checklist

Each item is a verb. Where a choice depends on your stack, we give the principle, not a fake-specific answer.

Assemble input and own the anchor table.

- `obs[]` come from decoding `@data/` frames off your Border Router. Each is one node's sighting of one device: `node_id`, `mac_hash`, `band`, `rssi`, `ts_ms`.
- `anchors[]` is yours to maintain (not on the wire): map `node_id` to surveyed `lat / lon`, with an `enrolled` flag and optional per-anchor path-loss overrides (`path_loss_a_cdb`, `path_loss_n_milli`; pass `<= 0` to fall back to the band default). A position is only physically meaningful with real surveyed coordinates; placeholders give output that is internally consistent but geographically meaningless.

- **Windowing is yours.** The library never subdivides by time; it uses `ts_ms` only to pick the latest sample per node and age-weight within a group. One call processes exactly the one batch you hand it (last 10 seconds, say). Same input in, same output out, no hidden state.
- **Respect the caps.** A single call processes at most `TS_MAX_GROUPS` (64) distinct `(mac_hash, band)` groups and at most `TS_MAX_NODES_PER_GROUP` (32) distinct nodes per group. Input beyond either bound is silently dropped by the core (no print, still `TS_OK` with a partial result). The CLI warns on stderr on over-cap input (still exits 0); a direct linker gets no warning, so keep each batch under the caps yourself or lose groups and nodes silently.

Persist positions. The library stores nothing; store any position you want after the call. Adapt to your stack:

- An **append-only history** of every position, each row carrying your own timestamp. Your audit trail and the input to any track filter.
- A **keep-latest current-state** record per `(mac_hash, band)`, so “where is this device now” is one lookup. Treat each position as a current-state gauge (best single estimate at that moment), not a sample to blindly average with older fixes; averaging raw positions mixes stale and fresh data and can smear a moving target. For smoothing use a track filter, not a naive mean.

Stamp time and handle staleness. The library has no clock and puts no time in output - the single most common integration mistake.

- Attach your own timestamp the moment you receive each position (system clock, or the observation time carried into the batch).
- Age out or reject stale fixes before acting. What counts as stale depends on your system and target speed; your call. Principle: prefer a fresh fix over a cached one, and never act on a fix so old it no longer describes reality.

Correlate over time. A real target produces a stream of fixes, not one point. The library gives one independent position per batch per `(mac_hash, band)` group and does not connect them across calls.

- Track per `(mac_hash, band)` - the identity the library correlates on within a batch and the natural key for stitching fixes into a track across batches.
- For smoothing or velocity, add a motion/track filter (Kalman, particle filter). Out of scope for the library by design; it stays pure and stateless.

Threshold on confidence and accuracy. Gate on `confidence` and `accuracy_m`.

- `confidence` is one of `40`, `70`, `85`: `40` (2-node bilateration) approximate and low-trust, `70` (3 anchors) a solid fix, `85` (4 or more anchors) high-trust. `accuracy_m` refines within a tier.
- There is no universal threshold. “Good enough” depends on your use case: a coarse presence signal tolerates a confidence-40 fix; a dispatch decision may demand `85` and a tight `accuracy_m`. Pick the gate fitting your consequence of being wrong. We prescribe no number, because any number would be wrong for some deployments.

Render on a map.

- Plot `lat / lon`. Swap to `[lon, lat]` if your mapping stack expects that order.
- For 3-or-more-anchor fixes the ellipse is an isotropic placeholder (`error_semi_major_m == error_semi_minor_m == accuracy_m`, bearing `0`). Draw as a circle of radius `accuracy_m`, or the ellipse primitive; they coincide.
- For a 2-node (confidence-40) fix, render honestly given mixed semantics. The along-baseline axis is a real roughly 1-sigma range uncertainty. The across-baseline axis is a hard mirror-ambiguity bound, not a confidence radius: it says how far the true position could sit on the far side of the baseline, not a Gaussian spread. A tidy 1-sigma circle overstates certainty; prefer a representation communicating the ambiguity (full ellipse, or a marker plus an explicit ambiguity note), not a small confident dot.
- Mapping libraries generally lack a native ellipse primitive. Approximate with a polygon: sample points around the ellipse using `semi_major`, `semi_minor`, `error_major_axis_deg` (bearing, 0 = North, clockwise) and draw it.

Alert and act. If you raise alerts or automate action:

- Dedupe. A single target produces a stream of fixes; naive per-fix alerting spams the operator. Debounce/dedupe per `(mac_hash, band)` track so one target is one alert (updated), not one per batch.
- Decide what a low-confidence or no-fix result means for your logic. A confidence-40 fix may warrant “possible contact, low trust” rather than a full alert. A target that stops producing any fix may be worth surfacing, or may just be out of range; that interpretation is yours.

Monitor no-output. The library returns `TS_OK` with zero positions for legitimate reasons (too few resolvable anchors, degenerate geometry); a silent “no output” must not be mistaken for “nothing is out there.”

- Log when a batch you expected to yield a fix yields none, and why if you can tell (fewer than 2 resolvable anchors, or rejected geometry).

- Alert yourself when anchors drop out or geometry degrades, so a coverage gap is visible instead of looking like a quiet area.

5. “No position” and “low confidence”: detect and do

Not-happy paths are first-class outcomes, not errors: the call still returns `TS_OK`, so detect by inspecting output, not a status code. “Resolvable” = the observing node has a matching enrolled anchor entry in your table.

Case	Detect	Do
Fewer than 2 resolvable anchors: no position (silently dropped, not low-confidence)	A device with observations in your batch has no entry in returned positions	Treat as “not enough coverage to locate,” not “not present.” If you expected a fix, check observing nodes are enrolled and present in your anchor table.
Degenerate/collinear geometry (3+ path only): no position (geometric-diversity gate rejects too-clustered or near-collinear participating anchors)	Same, a device with enough observations produces no returned position	Treat as a sensor-placement geometry problem, not a missing target; spread anchors out. Gate does not apply to the 2-node path (two points are trivially collinear).
Exactly 2 anchors: confidence-40 approximate fix. <code>accuracy_m</code> = ellipse semi-major axis (larger of along-baseline range noise and across-baseline mirror bound). Near-coincident pairs (closer than the 5 m minimum baseline) are dropped entirely rather than emit a falsely confident kilometre-scale ellipse.	<code>confidence == 40</code> and <code>measurement_count == 2</code>	Route as approximate/low-trust. Render honestly per section 4; do not treat its ellipse cross axis as a 1-sigma radius. Adding a third enrolled unit that sees the target auto-upgrades to a full triangulation fix with no other change.

6. Before you go live: integration checklist

- **Own the anchor table.** Maintain `node_id` to surveyed `lat / lon`, with `enrolled` and optional path-loss overrides. Use real surveyed coordinates.

- **Do your own windowing.** Decide which observations form one batch. The library does not subdivide by time.
- **Respect the caps.** Keep each batch under `TS_MAX_GROUPS` (64) and `TS_MAX_NODES_PER_GROUP` (32). If you link directly, enforce it yourself; the core drops overflow silently.
- **Stamp time.** Attach your own timestamp on receipt. The output has none.
- **Persist.** Store an append-only history plus a keep-latest current state per `(mac_hash, band)`. Treat each fix as a gauge, not something to average blindly.
- **Age out staleness.** Reject or expire old fixes before acting.
- **Correlate over time.** Track per `(mac_hash, band)`; add a motion filter if you need smoothing.
- **Threshold.** Gate on `confidence` (40 / 70 / 85) and `accuracy_m` for your use case. No universal threshold.
- **Render honestly.** Plot `lat` / `lon` (swap to `lon` / `lat` if needed). Draw the ellipse as a polygon. Do not draw the 2-node cross axis as a confidence circle.
- **Dedupe alerts.** One target is one alert, not one per batch.
- **Monitor no-output.** A `TS_OK` with zero positions can mean bad geometry or too few nodes. Do not mistake silence for “nothing there.”